

Architettura Degli Elaboratori

- Prima parte -

Rappresentazione dell'informazione

I calcolatori utilizzano un sistema di rappresentazione dell'informazione che utilizza solo due cifre, chiamate bit (0 e 1). Più bit vengono combinati tra di loro, più è complessa l'informazione. Con n bit si avranno 2^n combinazioni.

- byte: 8 bit
- nybble: 4 bit
- word: 32 bit
- halfword: 16 bit
- doubleword: 64 bit

Una rappresentazione è un modo per descrivere un'entità; infatti i sistemi numerici sono formati da:

1. l'entità numero o valore
2. la rappresentazione

Esempio:

'Sedici' può essere rappresentato in diversi sistemi, ne vediamo due:

1. decimale: 16
2. binario: 10000

Notiamo che l'entità è la stessa ma cambia la rappresentazione.

I sistemi numerici possono essere posizionali o non posizionali.

In un sistema numerico posizionale ogni cifra assume un valore diverso in base alla posizione che assume all'interno nel numero; un esempio è il sistema decimale ($12 \neq 21$).

Invece un sistema numerico non posizionale è il sistema romano; ogni cifra ha un valore e non cambia in base alla sua posizione; ad esempio se si scrivessero i seguenti valori: VI e IV, si noterebbe che cambia l'entità numero, ma le cifre che compongono il valore, anche se in posizioni diverse, hanno sempre lo stesso valore.

Nei sistemi posizionali un generico valore N è caratterizzato nella seguente rappresentazione:

$$N = d_{n-1}d_{n-2} \dots d_1d_0, d_{-1} \dots d_{-m}$$

$$N = d_{n-1} \cdot r^{n-1} + \dots + d_0 \cdot r^0 + d_{-1} \cdot r^{-1} + \dots + d_{-m} \cdot r^{-m}$$

$$N = \sum_{i=-m}^{n-1} d_i \cdot r^i$$

Esempio:

$$\begin{aligned} N &= 123,45 \\ &= 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \end{aligned}$$

Legenda:

1. $d \rightarrow$ singola cifra
2. $r \rightarrow$ base del sistema
3. $n \rightarrow$ numero di cifre della parte intera
4. $m \rightarrow$ numero di cifre della parte frazionaria

Se facessi il procedimento precedente su un numero espresso in binario (o una qualsiasi base), questo verrebbe convertito in decimale:

Esempi:

$$\begin{aligned} N &= 101 \text{ (base 2)} \\ &= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5 \end{aligned}$$

$$\begin{aligned} N &= 127 \text{ (base 8)} \\ &= 1 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 = 87 \end{aligned}$$

$$\begin{aligned} N &= A1 \text{ (base 16)} \\ &= A \times 16^1 + 1 \times 8^0 = 161 \end{aligned}$$

Nota bene: il pedice della base 16 ha diverse rappresentazioni, ad esempio H oppure davanti al numero si potrebbe trovare 0x.

$$A_H = 10_{10} = 1010_2$$

$$B_H = 11_{10} = 1011_2$$

$$C_H = 12_{10} = 1100_2$$

$$D_H = 13_{10} = 1101_2$$

$$E_H = 14_{10} = 1110_2$$

$$F_H = 15_{10} = 1111_2$$

Rappresentazione valori A - F in esadecimale

In una sequenza di bit, ne distinguiamo due in particolare:

110100**10**

Most Significant Bit

Least Significant Bit

La conversione da base 10 ad una qualsiasi base r avviene nel seguente modo;

1. Dividiamo il numero N per la base r fino a quando l'ultimo quoziente è minore della base stessa r
2. Prendiamo l'ultimo quoziente e tutti i resti delle divisioni e, procedendo dall'ultimo resto al primo, li scriviamo da sinistra a destra.

Esempio:

N = 12 (base 10) → in base 2

12 : 2 = 6	R = 0
6 : 2 = 3	R = 0
3 : 2 = 1	R = 1
1 : 2 = 0	R = 1

Quindi: 12 → 1100

N = 120 (base 10) → in base 8

120 : 8 = 15	R = 0
15 : 8 = 1	R = 7
1 : 8 = 0	R = 1

Quindi: 120 → 170

N = 1253 (base 10) → in base 16

1253 : 16 = 78	R = 5
78 : 16 = 4	R = 14 = E
4 : 16 = 0	R = 4

Quindi: 1253 → 4E5

Conversione da base 2 alla base 16/8 e viceversa viene fatta attraverso una fase di raggruppamento di bit.

Esempio:

0111 1111 0001 1010 $\rightarrow$ 7 15 1 10 $\rightarrow$ 7F1A

- 1010 (conversione da binario a decimale) = 10 in esadecimale corrisponde ad A
- 0001 (conversione da binario a decimale) = 1 in esadecimale corrisponde ad 1
- 1111 (conversione da binario a decimale) = 15 in esadecimale corrisponde ad F
- 0111 (conversione da binario a decimale) = 7 in esadecimale corrisponde ad 7

Convertire il numero AB216 in binario.

- A16=1010=10102
- B16=1110=10112
- 216=210=00102

Quindi $\rightarrow$ AB216 = 1010101100102

Un fenomeno che si potrebbe verificare è l'overflow; tale fenomeno avviene quando si è nell'impossibilità di rappresentare il risultato di una operazione (e.g., una somma o una sottrazione) con il numero di cifre a disposizione.

1. Kilo $\rightarrow 2^{10} = 1024$
2. Mega $\rightarrow 2^{20}$
3. Giga $\rightarrow 2^{30}$

Operazioni aritmetiche

Somma di due numeri binari:

Carry $\rightarrow$ 1 1 1
Addendo $\rightarrow$ 0 1 0 0 1 1
Addendo $\rightarrow$ 0 1 0 0 0 1
Somma $\rightarrow$ 1 0 0 1 0 0

Differenza di due numeri binari; per il prestito: il minuendo diventa 0 e prestito 2

Borrow $\rightarrow$ 2 2 2
Minuendo $\rightarrow$ 1 1 1 0 1
Sottraendo $\rightarrow$ 0 1 1 1 0
Differenza $\rightarrow$ 0 1 1 1 1

Overflow:

1. somma: non posso rappresentare un numero con n bit perché n bit sono pochi
2. differenza: non posso rappresentare un numero negativo

Rappresentazione numeri negativi

1. Modulo e Segno

Attraverso la rappresentazione di un numero binario con n bit, è possibile riservare l'n-esimo bit per indicare se il numero è positivo (0) oppure negativo (1).

Esempio:

1 0 0 0 1 0 0

Segno negativo
Valore assoluto

Quanti numeri posso rappresentare con n bit:

$$[- (2^{(n-1)} - 1) , (2^{(n-1)} - 1)]$$

Problema: Spreco di un bit per rappresentare il numero zero

0 0 0 0 oppure 1 0 0 0

2. Complemento a 1

Si utilizza un'operazione di complemento, ovvero si associa a ciascun bit il suo opposto.

$$0 \rightarrow 1 \quad 1 \rightarrow 0$$

Tutto ciò se e solo se il numero che si vuole rappresentare è negativo.

Esempio:

$$\begin{aligned} 3 \text{ (base 10)} &\rightarrow 0011 \\ -3 \text{ (base 10)} &\rightarrow 0011 \rightarrow 1100 \end{aligned}$$

Il problema dello 0 persiste.

3. Complemento a 2

Se il numero da rappresentare è un numero positivo, come nei precedenti casi, non ci sono problemi di rappresentazione, esse rimane invariato.

Altrimenti si applica la seguente metodologia

$-7 \rightarrow 0111 \rightarrow 1001$

Cosa è stato fatto:

1. scrivo il valore assoluto del numero in binario
2. parto da dx e trascrivo tutti gli 0 finché incontro un 1, a quel punto lo trascrivo e poi complemento

Problema dello 0 risolto.

Quanti numeri posso rappresentare con n bit:

$[-(2^{n-1}), (2^{n-1} - 1)]$

Operazioni aritmetiche 2

1. Modulo e segno

Somma:

1. segni concordi: tutto ok
2. segni discordi: il segno sarà quello del numero con valore assoluto maggiore e poi si applica una differenza bit a bit

Sottrazione:

1. segni concordi: tutto ok
2. segni discordi: segno del minuendo e poi somma bit a bit

Overflow

1. somma segno concorde
2. sottrazione segno discorde

2. Complemento a 2

Somma

1. sommo i bit compreso il segno
2. il riporto oltre gli n bit, viene scartato

Differenza:

1. trasforma in somma: $A-B \rightarrow A + CA2(B)$

Esempio:

+8 → 0000 1000 0000 1000
- (+5) → 0000 0101 → 1111 1011
 1 0000 0011

Cambio di rappresentazione da n a m bit:

1. se positivo il numero, aggiungo 0 davanti
2. se negativo il numero, aggiungo 1 davanti

Overflow:

1. $(+A) + (+B) = - C$
2. $(-A) + (-B) = + C$

Rappresentazione eccesso 128

Un numero X è rappresentato come $X + 128$

Esempio:

$X = 5 \rightarrow 5 + 128 = 133 \rightarrow 1000\ 0101$

Passando da CA2

$-3 \rightarrow 0000\ 0011 \rightarrow 1111\ 1101 \rightarrow 0111\ 1101$

Viene cambiato il Most Significant Bit

Cambio di convenzione: 0 negativi, 1 positivi

Rappresentazione dei numeri reali

1. Virgola fissa

Si riservano n bit per la parte intera e m bit per la parte decimale.

Se si parla di **unsigned** allora il segno non è presente nella rappresentazione del numero; se si parla di **signed** allora si avrà un bit dedicato al segno.

Parte intera: $I < N - 1$

Parte decimale: $D = N - (I + 1)$

Intervallo dei numeri rappresentabili: $[-2^{l-1-1}, 2^{l-1-1}]$

Esercizio:

$$101.01 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 4 + 0 + 1 + 0 + 0.25 = 5.25$$

Esercizio:

Rappresentare il numero 23.625 in virgola fissa con 8 bit per parte decimale e 8 bit per parte intera

segno + = 0
23 = 0010111

$$\begin{aligned} 0.625 \cdot 2 &= 1.250 \\ 0.250 \cdot 2 &= 0.500 \\ 0.500 \cdot 2 &= 1.000 \end{aligned}$$

Risultato $\rightarrow$ 0 0010111.10100000

2. Virgola mobile

I numeri in virgola mobile vengono espressi nel seguente formato:

$$N = \pm M \cdot B^E$$

M $\rightarrow$ mantissa
B $\rightarrow$ base
E $\rightarrow$ esponente

$363,4 \cdot 10^{34} \rightarrow$ rappresentazione non normalizzata

$3,634 \cdot 10^{36} \rightarrow$ rappresentazione normalizzata

Bit a disposizione	Segno	Esponente	Mantissa
32	1	8	23
64	1	11	52

Esercizio:

-53.5
segno - = 1

$$53 = 00110101$$

$$0.5 \cdot 2 = 1.00$$

$$110101.1 \rightarrow 1.101011 \cdot 2^5$$

1	00000101	101011000000000000000000
---	----------	--------------------------

Standard IEEE 754

Specifica il formato, le operazioni, le conversioni tra i diversi formati floating point e quelle tra i diversi sistemi di numerazione, il trattamento delle eccezioni.

$$X = (-1)^S \cdot (1 + 0.M) \cdot 2^E - 127$$

Esercizio:

Quale numero rappresenta ciò → 1 10000001 010000000000000000000000

Segno → -

Esponente → $129 - 127 = 2$

Mantissa → $01 = 1 + 2^{-2} = 1.25$

$$X = 1.25 \cdot 2^2 = 5 \rightarrow -5$$

Esercizio:

Rappresenta 8.5

Segno → +

Parte intera → $8 = 00001000$

Parte decimale → $0.5 \rightarrow 1$

$1000.1 \rightarrow 1.0001 \cdot 2^3$

Esponente → $3 + 127 = 130$ (10000010)

0	10000010	000100000000000000000000
---	----------	--------------------------

Esercizio:

Numero composto da 8 cifre esadecimali considerare la sua rappresentazione binaria e interpretarli come rappresentazione di due numeri razionali con standard IEEE 754:

Numero in floating point = 0 **100 0001 0** 011 0100 0000 0000 0000 0000

Segno → +

Esponente → $130 - 127 = 3$

Mantissa → 01101

$$\text{Numero: } 1011.01 = 11 + 2^{-2} = 11.25$$

Errore assoluto e relativo

Rappresentando un numero reale in virgola mobile si commette un errore di approssimazione.

In realtà viene rappresentato un numero razionale n' con un numero limitato di cifre significative:

ERRORE ASSOLUTO: $eA = n - n'$

ERRORE RELATIVO: $eR = eA / n = (n - n') / n$

Rappresentazione di caratteri

Possiamo associare a ciascun carattere, segno di interpunzione un numero. Possiamo rappresentare ciò attraverso la codifica ASCII e UNICODE.

1. ASCII standard: 1 carattere rappresentato con 7 bit
2. ASCII estesa: 1 carattere rappresentato con 8 bit
3. UNICODE: 1 carattere rappresentato con un numero tra 8 e 32 bit

La codifica UNICODE, al contrario di quella ASCII, considera i caratteri internazionali. Un carattere unicode rappresentato con 8 cifre esadecimali. 4 MLD di caratteri possibili.

Il fatto che ogni carattere occupa 32 bit è uno svantaggio per i S.O e applicazioni che non riescono a gestire 32 bit per carattere.

Nasce quindi UTF-8 che è una codifica a lunghezza variabile.

I primi 128 caratteri di UNICODE, equivalenti a quelli ASCII, sono in codifica 'naturale' mentre gli altri sono codificati con due, tre o quattro valori a 8 bit.

Circuiti digitali

I circuiti logici sono realizzati su chip di silicio come circuiti integrati.

Sono composti da porte e da fili depositati su chip di silicio inseriti in un package e collegati all'esterno con dei pin.

Ci sono diversi gradi di integrazione:

1. SSI → 1-10 porte
2. MSI → 10-100 porte
3. LSI → 100-100.000 porte
4. VLSI → >100.000 porte

Possiamo distinguere i circuiti digitali in due categorie:

1. circuiti combinatori: lo stato delle uscite dipende solo dalla funzione logica applicata allo stato istantaneo delle sue entrate
2. circuiti sequenziali: lo stato delle sue uscite dipende sia dalla funzione logica applicata alle sue entrate sia a dati pregressi salvati in memoria.

Le porte logiche hanno, di base, n entrate e 1 sola uscita; ci permettono di svolgere operazioni primitive e derivate.

Porte logiche fondamentali: AND, OR, NOT

Porte logiche derivate: NAND, NOR, XOR

Componenti elettronici

Decoder: componente elettronico che ha n ingressi e 2^n uscite.

Lo scopo del decoder è quello di impostare allo stato alto l'uscita corrispondente alla conversione in base 10 del valore in ingresso in base 2.

Multiplexor: è un componente elettronico caratterizzato da:

1. 2^n entrate principali
2. n selettori
3. 1 uscita

Il valore del selettore determina quale input diviene output.

Un multiplexor sarà formato da n porte and e una porta or.

Possiamo implementare logiche a due livelli: PLA (programmable logic array) ovvero una somma di prodotti.

(Possiamo avere quindi più porte or)

La maggior parte delle operazioni vengono svolte su 32 bit, quindi si utilizzano i bus che permettono di trattare più linee in input come un unico segnale.

Un multiplexor con bus a 32 bit corrisponde ad un array di 32 multiplexor a 1 bit.

Arithmetic Logic Unit

ALU (arithmetic logic unit) è la parte del processore che svolge operazioni aritmetico-logiche.

Addizione: presente un carryIn e un carryOut

Sottrazione: la otteniamo, per CA2, negando il valore del sottraendo.

Leggi de Morgan:

1. $\text{NOT}(A \text{ OR } B) \rightarrow \text{NOT}(A) \text{ AND } \text{NOT}(B)$
2. $\text{NOT}(A \text{ AND } B) \rightarrow \text{NOT}(A) \text{ OR } \text{NOT}(B)$

Per ottenere un ALU a 32 bit, ho in sequenza 32 ALU da 1 bit.

SLT (set on less then) $\rightarrow$ se $a < b$ risultato 1 altrimenti 0

$a < b \longleftrightarrow a - b < 0$ e bit 31 dell'alu a 32 bit è 1;

per realizzare ciò si effettua una sottrazione.

La prima ALU riceve un carryIn (anche per sottrazione) = 1

Casi di overflow:

1. $a - b < 0$ e bit 31 alu = 0
2. $a - b > 0$ e bit 31 alu = 1

Si aggiunge l'overflow detection e il set, che è l'output che verrà dato come MSB alla stringa di bit ottenuta.

BEQ (branch-on-equal) $\rightarrow a = b$ sse $a - b = 0$ (usiamo ancora una sottrazione)

OR di tutte le uscite e poi nego il risultato.

Circuiti sequenziali

I circuiti sequenziali si distinguono in due tipologie:

1. asincroni (SR-Latch) che non fanno uso di clock
2. sincroni (Flip flop) che fanno uso di clock

I circuiti sequenziali sono formati da elementi di memoria e reti combinatorie che elaborano le informazioni.

SR-Latch organizza le porte per avere un elemento di memoria.

S = Set

R = Reset

La combinazione in input (0,0) viene detta di riposo perché mantiene il valore memorizzato in precedenza.

Se ci troviamo nella situazione in cui Q e NOT(Q) sono uguali, abbiamo una configurazione instabile.

L'output del circuito diventa stabile dopo che si sono attraversate un certo numero di porte e quindi dopo un certo tempo; bisogna quindi memorizzare gli output dopo questo intervallo di tempo.

Soluzione: si usa il clock.

Il clock è un segnale a gradino che ha un periodo (e una frequenza pari a $1/\text{Periodo}$), predeterminato e costante, abbastanza grande che assicura la stabilità degli output. Il clock determina il ritmo dei calcoli e delle relative operazioni di memorizzazione. Quindi il circuito diventa sincrono.

Il latch sincronizzato con il clock diventa un D-Latch.

Se $D=1 \rightarrow$ setting

Se $D=0 \rightarrow$ resetting

Se il clock è deasserted ($S=R=0$) non viene memorizzato nulla; mentre se il clock è asserted avviene la memorizzazione in funzione del valore D.

Il segnale D deve rimanere stabile prima, durante e dopo che il clock sia asserted; parliamo di hold time e setup time.

Come funziona il D-Latch:

Durante l'intervallo alto del clock, il valore D si propaga nel circuito e si ottiene il risultato Q che viene memorizzato; anche tutte le variazioni di D, quindi di Q, in intervallo alto del clock vengono memorizzate.

Quando il clock torna a 0, allora Q si stabilizza e non viene memorizzato nulla.

Parliamo di **trasparenza del latch**.

Fino ad ora abbiamo usato una tipologia di timing chiamata level-triggered, che si basa su intervallo alto/basso del clock per memorizzare.

Ma se volessimo che lo stato ci servisse sia per memorizzare che per leggere i valori (quindi come input e come output), dovremmo utilizzare un'altra metodologia di timing: edge-triggered.

Edge-triggered memorizza sul fronte in salita/discesa del clock. La memorizzazione è istantanea e il segnale di ritorno 'sporco' quindi non fa in tempo ad arrivare.

Parliamo dunque del D Flip-Flop.

Il D flip flop utilizza due D latch in sequenza; quando il primo (master) è pronto a memorizzare il secondo sarà chiuso (NOT C). L'uscita del master sarà corrispondente a valore alto se $C=1$ e $D=1$, basso se $C=1$ e $D=0$; mentre se $C=0$ l'uscita si sarà stabilizzata al precedente valore memorizzato.

Quando il clock avrà segnale basso, si aprirà il secondo latch (slave). Il valore Q' messo in relazione con C negato. Se Q' alto allora segnale di Q alto, altrimenti basso. Quando C torna alto, Q si stabilizza.

Datapath e register file

Un registro è costituito da n flip flop.

Nel MIPS ogni registro è di 1 word; i registri sono organizzati nei register file.

Il register file del MIPS ha 32 registri (32 x 32 = 1024 flip flop).

Il register file permette la lettura di due registri e la scrittura di uno.

Il register file ha 5 ingressi e due uscite.

Nel Datapath della CPU il clock non entra direttamente nei vari flip-flop; viene messo in AND con un segnale di controllo "Write".

Il segnale Write determina se, in corrispondenza del fronte di discesa del clock, il valore D debba (o meno) essere memorizzato nel registro.

Read Reg1 # (5 bit)

numero del 1o registro da leggere

Read Reg2 # (5 bit)

numero del 2o registro da leggere

Read data 1 (32 bit)

valore del 1o registro, letto sulla base di **Read Reg1 #**

Read data 2 (32 bit)

valore del 2o registro, letto sulla base di **Read Reg2 #**

Write Reg # (5 bit)

numero del registro da scrivere

Write data (32 bit)

valore da scrivere nel registro **Write Reg #**

Lettura dal register file: utilizza due segnali Read reg1 e reg2, usa due mux da 32 ingressi con 1 segnale di controllo; il register file fornisce sempre una coppia di registri in output.

Scrittura nel register file: utilizza tre segnali (1. registro da scrivere (register number), 2. il valore da scrivere (register data), 3. il segnale di controllo). Usa un decoder che decodifica il numero del registro da scrivere. Il segnale write, già in and con il clock, viene messo in and con l'output del decoder. Se write non affermato, nessun valore verrà scritto.

Oltre alle piccole memorie implementate per mezzo di registri e file di registro, esistono altri tipi di memorie che possiamo distinguere in base a diversi parametri:

1. **Dimensione:** quantità di dati memorizzabili
2. **Velocità:** l'intervallo di tempo tra la richiesta del dato e il momento in cui è disponibile
3. **Consumo:** potenza assorbita
4. **Costo:** costo per bit.

Idealmente un calcolatore dovrebbe avere quanta più memoria possibile, ad alta velocità, basso consumo e minimo costo.

Memorie

Rom (read only memory) → circuito combinatorio in cui ogni ingresso (indirizzo) corrisponde ad un'uscita (contenuto della cella di memoria con quell'indirizzo)

Le locazioni di memoria possono essere lette.

N bit in input, una tra le 2^n celle di memoria.

PROM → programmabili se il designer conosce il contenuto

EPROM → possono essere cancellate con un processo lento a luce ultra violetta

Ram (random access memory) → insieme di celle (1 byte) ognuna individuata da un indirizzo. È una memoria volatile, ovvero perde il proprio contenuto quando si spegne; è la memoria principale; è meno veloce della memoria dei registri, ma più capiente.

Operazioni: lettura e scrittura. Con n bit si indirizzano 2^n celle di memoria

Lettura: il contenuto della cella di memoria indirizzata dal registro indirizzi è copiato nel registro dati.

Scrittura: il contenuto del registro dati è copiato nella cella di memoria indirizzata nel registro indirizzi.

SRAM (static ram) → è più veloce

- . realizzata con latch

- . tempi di accesso di 0,5 - 2,5 ns

- . SRAM a basso consumo (5-10 volte più lente)

viene realizzata come una matrice H (altezza → numero di celle) x W (larghezza → numero di latch)

Il singolo indirizzo è usato sia in lettura che in scrittura; non possibile leggere e scrivere contemporaneamente

DRAM (dynamic ram) → è più capiente ma più lenta

- . ogni bit è memorizzato tramite un condensatore

- . tempi di accesso intorno a 50 - 70 ns

- . è necessario rinfrescare il contenuto a intervalli di tempo prefissati

SSRAM - DSRAM (Synchronous) → aumentano la banda di trasferimento sfruttando la seguente proprietà:

suddivisione in due blocchi dell'indirizzo

1. parte alta per accedere alla riga
2. parte bassa per accedere alla colonna

Macchine a stati finiti (FSM)

Le FSM vengono usate per la descrizione di circuiti sequenziali, sono composte da un set di due funzioni:

1. next state function: determina lo stato successivo in base allo stato corrente e ai valori in ingresso
2. output function: produce dei risultati partendo dallo stato corrente e valori in ingresso

Sono sincronizzate con il clock.

Assembly

Linguaggio macchina → comprensibile direttamente dal calcolatore, quindi è binario; scrittura lunga del codice e facile commettere errori.

Linguaggio assembler → rappresentazione simbolica del linguaggio macchina, è più comprensibile perché usa i simboli e non i bit; deve essere tradotto dall'assemblatore

Debugger → aiuta a trovare i bug eseguendo il programma in modo controllato.

1. esecuzione passo passo del programma
2. ispeziona valori delle variabili ed espressioni
3. interruzione nei breakpoint
4. interruzione nel caso di modifica di valori di variabili (watchpoint)
5. visualizzazione degli indirizzi di memoria delle variabili o delle istruzioni

Compilatore → traduzione di un linguaggio ad alto livello al linguaggio assembly; dopo questa fase, l'assemblatore traduce da assembly a linguaggio macchina.

Spesso con il termine compilazione si intendono entrambi i passaggi perché alcuni compilatori includono l'assemblatore.

Assembler → converte da linguaggio assembler a linguaggio macchina; gestisce le etichette, pseudo istruzioni e numeri in basi diverse.

Processo di assemblaggio → processo che esamina riga per riga il codice sorgente assembly e traduce in linguaggio macchina.

I passi importanti sono due:

1. Traduce i codici mnemonici delle istruzioni in binario
2. Traduce i riferimenti simbolici (variabili, registri, etichette di salto ecc) nei corrispondenti indirizzi numerici

Visto che le etichette di salto generano problemi con altre etichette, bisogna che il programma venga letto due volte (due passi, infatti si dice che l'assemblatore sia un traduttore a due passi). Ogni modulo assemblato di default parte dall'indirizzo 0.

Tabella dei simboli → contiene tutti i riferimenti simbolici del modulo che si sta traducendo.

Linker → inserisce in memoria, in modo simbolico, il codice dei moduli dati; determina gli indirizzi dei dati e delle etichette che compaiono nelle istruzioni; corregge riferimenti interni, esterni e risolve i riferimenti in sospeso. Genera il file eseguibile.

Loader → legge l'intestazione del file eseguibile per stabilire la lunghezza delle istruzioni e delle variabili e crea lo spazio per contenerle.

Copia istruzioni e dati in memoria; copia nello stack eventuali dati passati al programma principale.

Inizializzazione dei registri e dello stack che deve puntare alla prima locazione libera

Procedura di startup → copia i parametri nei registri argomento e chiama la procedura principale del programma; dopo che la procedura principale ha finito, startup riprende il controllo e termina il programma con exit.

Direttive all'assemblatore

.data <addr>
quello che segue va nel segmento dati

.byte b1, ..., bn
inizializza i valori in byte successivi

.word w1, ..., wn
inizializza i valori in word successive

.text <addr>
quello che segue va nel segmento text

ori → fa or bit a bit tra un registro e immediate

Manipolazione delle costanti

carica una costante in registro
lui rt, imm → load upper immediate (16 bit la immediate)
possibile anche 32 bit → i bit bassi saranno 0

lui \$s0, 61
contenuto di \$s0 → 0000 0000 0011 1101 0000 0000 0000 0000